

PULSO.AI
2026

Programa de Fluência

Módulo 3 • Trilha Tech

Agentic AI Deep Dive: ferramentas, Bedrock AgentCore, orquestração e confiança.



Agentic AI Deep Dive

Agentes · Ferramentas e Strands · Bedrock AgentCore · Orquestração ·
Confiança

Módulo 3 · Trilha Tech · Programa de Fluência em IA

TRILHA TECH

OUSE **PENSAR** DIFERENTE

JORNADA DO PROGRAMA

O que o seu acelerador ganha em cada encontro



COMO VAMOS TRABALHAR HOJE

Conteúdo, duas dinâmicas e demonstração ao vivo

Abertura Onde paramos ontem10 min	Orquestração Vários agentes, humano no loop e onde falham20 min
O que é um agente Com a dinâmica 1: você é o agente, na tela30 min	Confiança Observar, avaliar e limitar20 min
Ferramentas: as mãos do agente Strands, Text-to-SQL e MCP25 min	Dinâmica 2 • Kahoot Revisão do dia10 min
AgentCore: a plataforma Pilares, Gateway, Policy e harness40 min	Demonstração ao vivo Tudo o que vimos, testado na central55 min
Intervalo No meio da manhã20 min	Fechamento O dever de casa e o Checkpoint10 min

O QUE VOCÊ LEVA HOJE

Cinco coisas que você vai saber fazer

1

Explicar o que é um agente

E a diferença para um assistente e para um fluxo fixo.

2

Desenhar boas ferramentas

Nome, descrição, parâmetros: o que faz um agente acertar.

3

Situar cada peça do AgentCore

Onde roda, como lembra, quem autoriza, como se observa.

4

Escolher a orquestração

Um agente, vários agentes, e onde entra o humano.

5

Decidir quando não usar agente

A decisão de arquitetura que mais economiza tempo.

ENTREGÁVEL

O projeto da central (agente, ferramentas e interface), pronto para receber as ferramentas da sua área.

ONDE PARAMOS

O assistente entendeu tudo. E a Dona Helena continua esperando.

O QUE FIZEMOS ONTEM

- ✓ Três assuntos identificados na mensagem
- ✓ JSON confiável, com urgência e categoria
- ✓ Dúvida do jejum respondida com o protocolo (RAG)
- ✓ Rascunho protegido por Guardrails

O QUE AINDA É MANUAL

- 🔍 Abrir a agenda e achar a consulta de quinta
- 🕒 Procurar horário livre na cardiologia
- 📖 Registrar a mudança de convênio
- 📍 Remarcar e avisar a paciente

O modelo leu, consultou o protocolo e escreveu. Quem abriu os sistemas foi uma pessoa. História fictícia, dados sintéticos.

A PERGUNTA DO DIA

E se o assistente pudesse agir?

Consultar, decidir o próximo passo, executar e prestar contas.

DUAS FORMAS DE TRABALHAR

Do assistente que responde ao agente que resolve

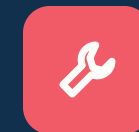


Assistente

Recebe um texto, devolve um texto.

Uma chamada ao modelo. Previsível, barato, rápido.

`"Aqui está um rascunho de resposta para a Helena."`



Agente

Recebe um objetivo, decide os passos e usa ferramentas até cumprir.

Várias chamadas ao modelo. Flexível, mais caro, mais lento.

`"Consulta remarcada para terça, 9h. Preparo do exame enviado."`

01

O que é um agente

Modelo, ferramentas e um loop. Nada além disso.

30 MINUTOS

Você é o agente

O PEDIDO

“Preciso remarcar minha consulta de quinta com o cardiologista. E o exame de sangue que ele pediu precisa de jejum?”

COMO VAI SER

- 1 A turma é o modelo**
Decide em voz alta qual ferramenta pedir, uma por vez.
- 2 O instrutor é o código**
Executa e mostra o resultado na tela.
- 3 Repete até responder**
Contem as voltas.

NA MESA: SEIS FERRAMENTAS

buscar_consultas consultas de um paciente	buscar_horarios vagas por especialidade
consultar_preparo preparo de um exame	remarcar_consulta altera a agenda
consultar_clima previsão do tempo	abrir_chamado_ti chamado para o service desk

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE

`buscar_consultas`

Cardiologia, Dr. Moura, quinta 14h
(consulta C-1001).

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE

`buscar_consultas`

Cardiologia, Dr. Moura, quinta 14h
(consulta C-1001).

DEVOLVE

`buscar_horarios`

Terça 9h · quarta 16h30 · sexta 8h.

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE

buscar_consultas

Cardiologia, Dr. Moura, quinta 14h
(consulta C-1001).

DEVOLVE

buscar_horarios

Terça 9h · quarta 16h30 · sexta 8h.

DEVOLVE

consultar_preparo

Glicemia: jejum de 8 horas.
Hemograma: sem jejum.

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE

buscar_consultas

Cardiologia, Dr. Moura, quinta 14h
(consulta C-1001).

DEVOLVE

buscar_horarios

Terça 9h · quarta 16h30 · sexta 8h.

DEVOLVE

consultar_preparo

Glicemia: jejum de 8 horas.
Hemograma: sem jejum.

DEVOLVE

remarcar_consulta

PAUSA: esta ação altera a agenda.
Quem aprova?

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE

buscar_consultas

Cardiologia, Dr. Moura, quinta 14h
(consulta C-1001).

DEVOLVE

buscar_horarios

Terça 9h · quarta 16h30 · sexta 8h.

DEVOLVE

consultar_preparo

Glicemia: jejum de 8 horas.
Hemograma: sem jejum.

DEVOLVE

remarcar_consulta

PAUSA: esta ação altera a agenda.
Quem aprova?

DEVOLVE

consultar_clima

Sol em Belo Horizonte, 27 °C. Serve
para quê?

DINÂMICA 1 · OS RESULTADOS

O que cada ferramenta devolve

DEVOLVE buscar_consultas Cardiologia, Dr. Moura, quinta 14h (consulta C-1001).	DEVOLVE buscar_horarios Terça 9h · quarta 16h30 · sexta 8h.	DEVOLVE consultar_preparo Glicemia: jejum de 8 horas. Hemograma: sem jejum.
DEVOLVE remarcar_consulta PAUSA: esta ação altera a agenda. Quem aprova?	DEVOLVE consultar_clima Sol em Belo Horizonte, 27 °C. Serve para quê?	DEVOLVE abrir_chamado_ti Chamado aberto no service desk. Era isso mesmo?

A EQUAÇÃO

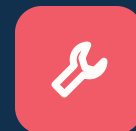
Agente = modelo + ferramentas + loop



Cérebro

O modelo de linguagem, que decide o próximo passo.

Módulo 2: escolha e prompt



Mãos

As ferramentas: funções que consultam e executam.

Hoje: bloco 2



Loop

Pensar, agir, observar o resultado e repetir.

Hoje: a seguir



Regras

Objetivo, limites e tom, no system prompt.

Módulo 2: as sete camadas



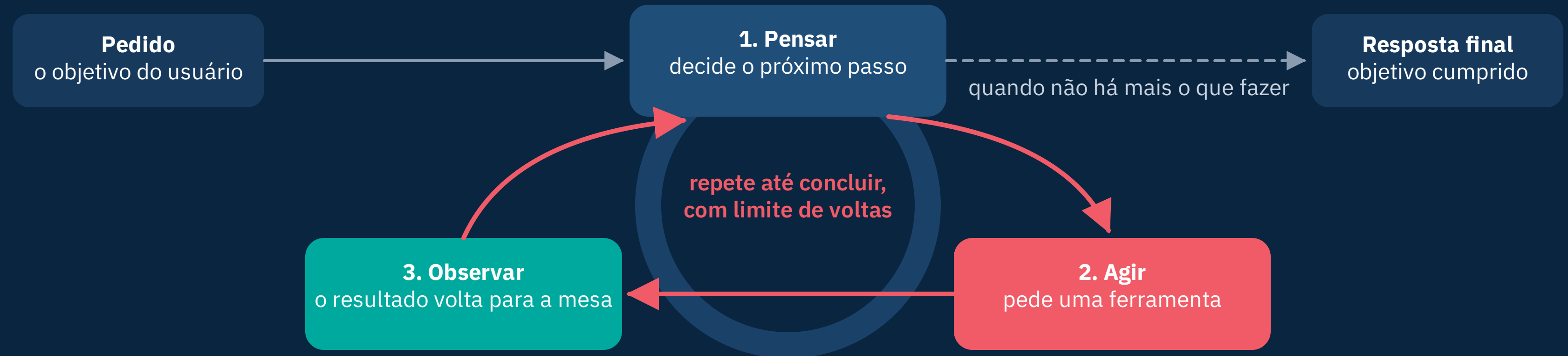
Memória

O que ele lembra da conversa e do usuário.

Hoje: bloco 3

O LOOP DO AGENTE

Pensar, agir, observar, repetir



O modelo não executa nada

Ele só escreve qual ferramenta quer usar. Quem executa é o seu código.

Cada volta é uma chamada

Com todo o histórico na mesa. Mais voltas, mais tokens, mais espera.

Todo loop precisa de freio

Número máximo de voltas, tempo e custo. Sem isso, um agente confuso roda sem parar.

O LOOP NA PRÁTICA

O pedido da Dona Helena, volta por volta

1	PENSA	“Preciso achar a consulta atual dela.”
2	AGE	buscar_consultas(paciente=“Helena Duarte”)
3	OBSERVA	Cardiologia, Dr. Moura, quinta 14h
4	AGE	buscar_horarios(especialidade=“cardiologia”, semana=“próxima”)
5	OBSERVA	Terça 9h, quarta 16h30, sexta 8h
6	AGE	consultar_preparo(exame=“hemograma e glicemia”)
7	OBSERVA	Glicemia: jejum de 8 horas. Hemograma: sem jejum
8	PENSA	“Remarcar muda a agenda: preciso de aprovação humana.”
9	RESPONDE	Propõe terça 9h, informa o jejum e sinaliza o convênio para o cadastro

A DECISÃO DE ARQUITETURA

Fluxo fixo ou agente?



Fluxo fixo (workflow)

O **seu código** define o caminho. O modelo entra em etapas específicas: classificar, extrair, redigir.

- ✓ Previsível e fácil de testar
- ✓ Mais barato e mais rápido
- ⚠ Não lida com o inesperado



Agente

O **modelo** define o caminho, escolhendo ferramentas conforme o que encontra.

- ✓ Lida com pedidos variados e abertos
- ✓ Menos código de orquestração
- ⚠ Mais caro, mais lento, mais difícil de testar

ESCADA DE AUTONOMIA

Suba um degrau só quando o anterior não resolver

- 4 Vários agentes** Especialistas que colaboram. Só quando um agente não dá conta.
- 3 Um agente com ferramentas** O modelo escolhe os passos. O tema de hoje.
- 2 Cadeia de chamadas** Classifica, depois extrai, depois redige. Código no comando.
- 1 Uma chamada bem feita** O acelerador do módulo 2, com prompt e RAG. Resolve mais do que parece.

Agentes trocam latência e custo por flexibilidade. Se o seu problema cabe em um fluxograma, fique nos degraus 1 e 2.

02

Ferramentas: as mãos do agente

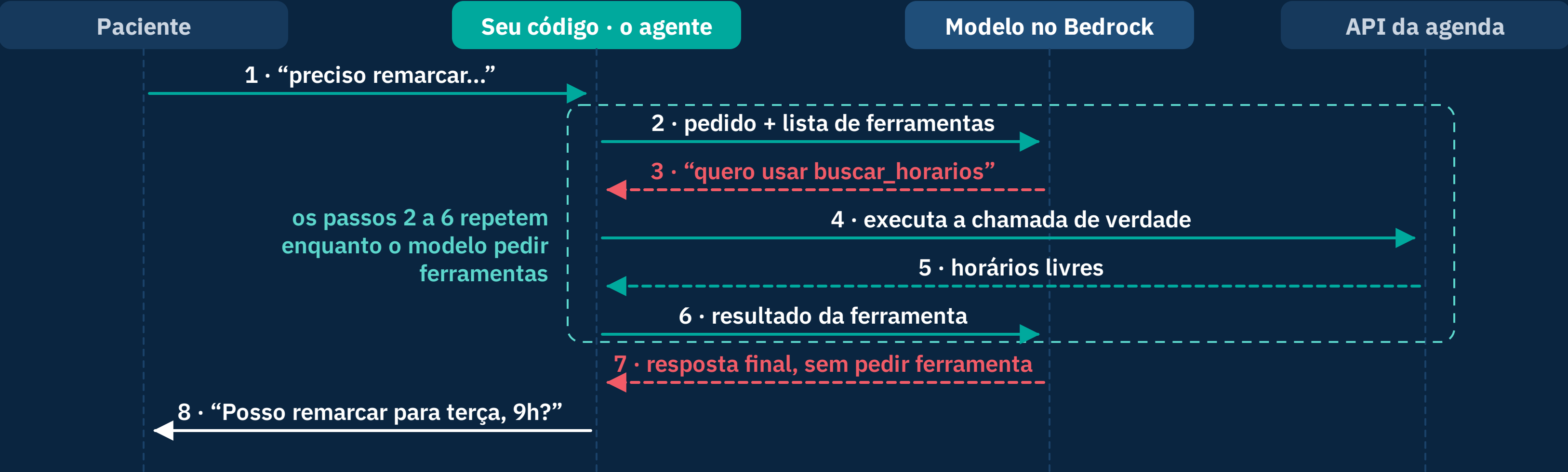
Como o modelo usa uma ferramenta, o Strands, boas práticas, Text-to-SQL e MCP.

25 MINUTOS

O MECANISMO

Você já viu em setembro: IAM e menor privilégio

Como o modelo “usa” uma ferramenta

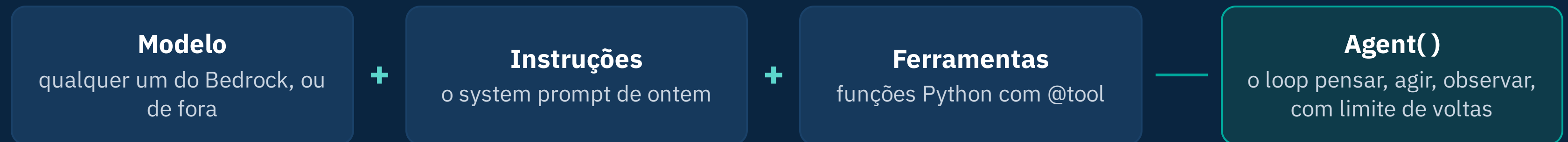


O modelo nunca toca nos seus sistemas: nenhuma seta sai dele para a API. Ele só escreve um pedido; quem executa é o seu código, com o seu IAM, e tudo fica no seu log.

ANTES DO CÓDIGO

Strands Agents: o que é e para que serve

Um SDK aberto da AWS, em Python e TypeScript, para construir agentes. Você entrega três coisas; ele entrega o loop pronto.



Para que serve

Ter um agente de verdade sem escrever o loop à mão, e trocar de modelo mudando uma linha.

Onde roda

Na sua máquina, numa Lambda, num container ou no AgentCore Runtime, sem mudar o código.

Não é obrigatório

É o caminho mais curto na AWS. LangGraph, CrewAI e OpenAI Agents SDK fazem o mesmo papel.

ANATOMIA DE UMA FERRAMENTA

Uma função comum, com uma boa descrição

```
from strands import tool

@tool
def buscar_horarios(especialidade: str,
                    semana: str) → list:
    """Lista horários livres para consulta.

    Use quando o paciente quiser marcar ou
    remarcar. Não use para exames.
    """
    return agenda.consultar(especialidade, semana)
```

- 1 **Nome** com verbo: diz o que a ferramenta faz.
- 2 **Parâmetros** poucos e com tipo claro.
- 3 **Descrição**: quando usar e quando não usar. É o prompt da ferramenta.
- 4 **Retorno** enxuto: só o que o modelo precisa para decidir.

BOAS PRÁTICAS

Boas ferramentas fazem bons agentes



Separe consulta de ação

Ler é seguro; alterar não. Ferramentas diferentes, permissões diferentes.



Erros que ensinam

“Especialidade não encontrada. Opções: cardiologia, ortopedia...” ajuda o agente a se corrigir.



Retorne pouco

Os 5 horários relevantes, não a agenda inteira. Cada caractere volta para a mesa do modelo.



Menor privilégio

Cada ferramenta com a role IAM mínima. O agente herda os limites das ferramentas.



Pense no estagiário

Um colega novo entenderia para que serve só lendo o nome e a descrição?



Teste sozinha

Uma ferramenta é uma função: teste unitário normal, sem modelo envolvido.

FERRAMENTAS · DADOS ESTRUTURADOS

Text-to-SQL: o agente pergunta ao banco



O CONTEXTO QUE O MODELO RECEBE

```
<schema> tabelas, colunas e tipos
  consultas(id, paciente_id, especialidade, status, data)
  pacientes(id, nome, convenio_id)
<relacoes> como as tabelas se ligam
  consultas.paciente_id → pacientes.id
<dicionario> o que cada valor quer dizer
  status: 'C' = cancelada · 'R' = realizada
<exemplos> perguntas reais com o SQL certo
  “consultas por especialidade no mês” → SELECT ...
```

DE ONDE VEM E COMO ENTRA

- 1 Fonte**
Comentários no banco, Glue Data Catalog, dicionário do time de dados.
- 2 Seleção**
Schema grande? Busca só as tabelas ligadas à pergunta.
- 3 Montagem**
No system prompt, entre tags, versionado como código.

TEXT-TO-SQL BEM FEITO

Duas formas na AWS, seis cuidados

COMO FAZER NA AWS

PRONTO

Knowledge Bases · dados estruturados

- Redshift ou Glue Catalog
- Descrição de tabelas
- Incluir ou excluir
- Consultas curadas
- Tempo máximo
- GenerateQuery: só o SQL

SOB MEDIDA

Sua ferramenta SQL

- Lambda + qualquer banco
- Postgres, Athena, SQL Server
- Publicada no Gateway (bloco 3)

SEIS CUIDADOS

- 1

Só leitura

Usuário sem escrita, sobre views.
- 2

Documente o banco

Tabelas, colunas, relações e regras.
- 3

Exemplos curados

Perguntas reais com o SQL certo.
- 4

Valide antes

Só SELECT, LIMIT e tempo máximo.
- 5

Mostre o SQL

Quem pergunta confere a consulta.
- 6

Permissão no banco

Dado de paciente: filtro por linha e coluna.

MCP

Um conector padrão para ferramentas

O **Model Context Protocol** é um padrão aberto para expor ferramentas a agentes. Escreveu uma vez, qualquer agente compatível usa.

É o USB-C das ferramentas: o mesmo plugue serve para o agente da central, para o assistente de código do time e para o que vier depois.

Padrão de mercado

Criado pela Anthropic, hoje mantido por uma fundação ligada à Linux Foundation, com AWS, Google, Microsoft e OpenAI.

Na AWS

O AgentCore Gateway fala MCP, o Runtime hospeda servidores MCP e existe um AWS MCP Server oficial desde maio de 2026.

Para vocês

Cada API interna exposta como MCP vira um ativo reutilizável da Rede, não uma integração de um projeto só.

03

AgentCore: a plataforma

Tudo que falta entre o protótipo e o agente em produção.

40 MINUTOS

DO PROTÓTIPO À PRODUÇÃO

As perguntas que alguém vai fazer antes do go-live

Onde isso roda, com isolamento entre usuários?	Runtime	Como garanto que ele não faz o que não deve?	Policy
Como ele alcança nossas APIs com segurança?	Gateway	Como vejo o que ele fez, passo a passo?	Observability
Como ele lembra do paciente na próxima conversa?	Memory	Como sei se ele está respondendo bem?	Evaluations
Em nome de quem ele está agindo?	Identity	Preciso mesmo escrever o loop?	harness

O AgentCore é um conjunto de serviços independentes. Dá para usar um só, ou todos, com qualquer framework e qualquer modelo.

OS PILARES

AgentCore: 13 peças, use só as que precisar

ONDE O AGENTE RODA

harness
O loop do agente, rodado pela AWS: você só declara.

Runtime
Sessão isolada para o seu código, qualquer framework.

O QUE ELE USA

Gateway
Suas APIs e Lambdas viram ferramentas MCP.

Browser
Um navegador na nuvem para usar sites.

Code Interpreter
Um lugar isolado para rodar código.

Payments
Paga APIs e conteúdo.

O QUE LEMBRA E PODE

Memory
O que lembrar da conversa e do usuário.

Identity
Em nome de quem ele age.

Policy
A regra que trava cada uso de ferramenta.

COMO SE ACOMPANHA

Observability
Cada passo registrado no CloudWatch.

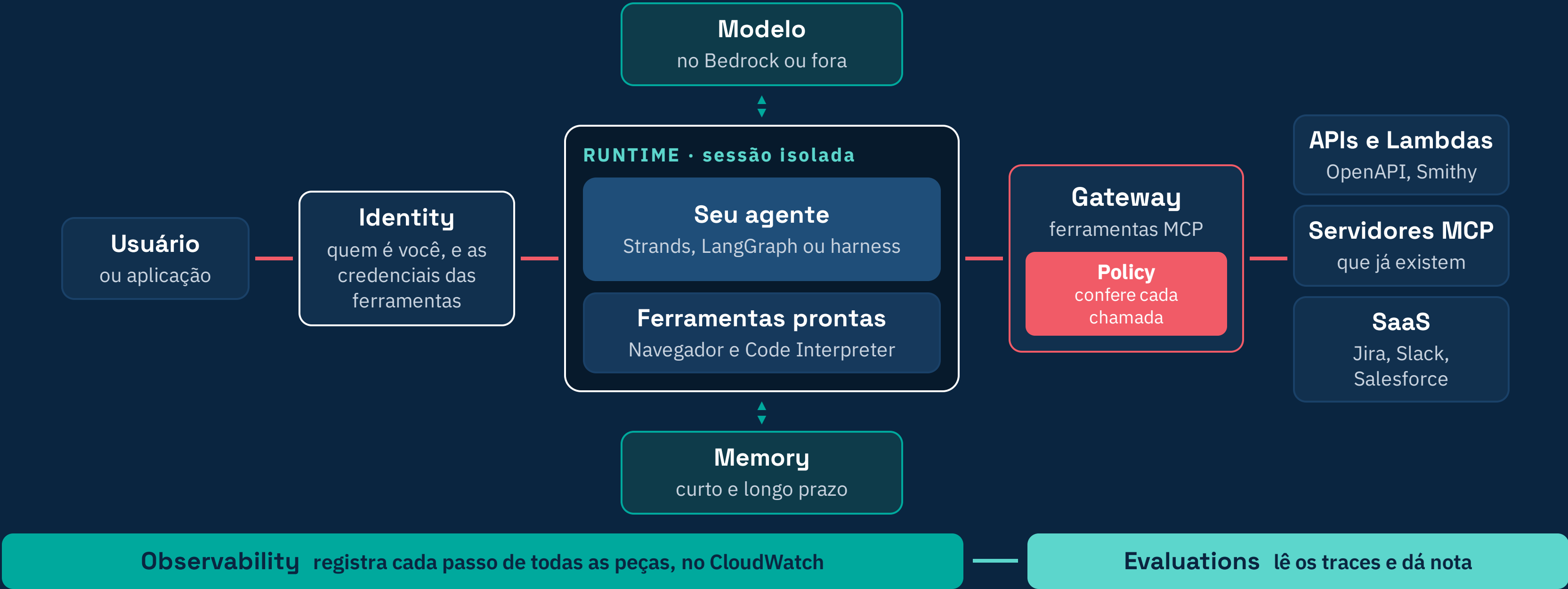
Evaluations
Nota automática de qualidade.

Optimization
Sugestões e testes A/B para melhorar.

Registry
Catálogo de agentes e ferramentas.

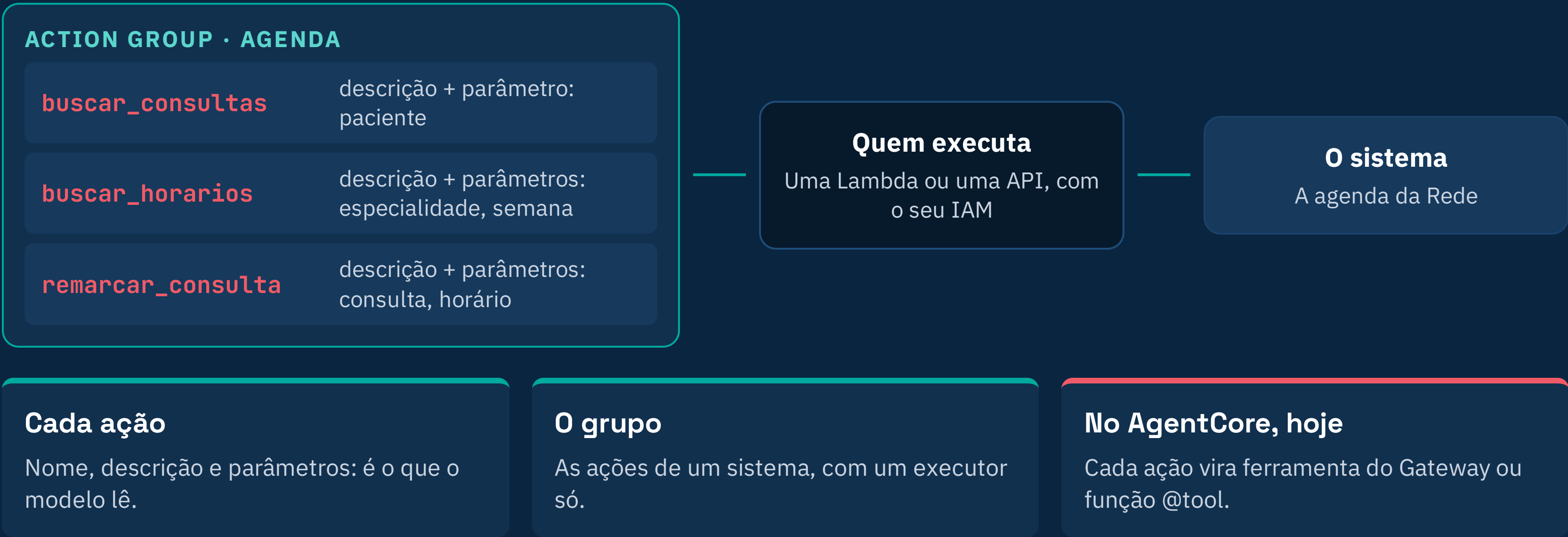
O MAPA

As peças do AgentCore, em volta do seu agente



O TERMO DA EMENTA

Action Group: o pacote de ações do agente



AGENTCORE GATEWAY

Suas APIs viram ferramentas, sem reescrever nada

APIs REST

A partir da especificação OpenAPI

Funções Lambda

O mesmo executor dos Action Groups

Servidores MCP

Os que já existirem

SaaS

Slack, Jira, Salesforce e outros

**Gateway**

Um endereço MCP único, com autenticação, limites de uso e catálogo

Agente da central

Descobre e usa as ferramentas

Outros agentes

Reaproveitam o mesmo catálogo

Com centenas de ferramentas, o Gateway oferece **busca semântica**: o agente recebe só as poucas relevantes para o pedido, em vez do catálogo inteiro.

AGENTCORE GATEWAY

Seis coisas que o Gateway faz por você

1 Conecta

Lambda, OpenAPI, Smithy, servidores MCP e integrações prontas: Salesforce, Slack, Jira, Asana, Zendesk.

2 Protege a entrada

Só entra quem tem token JWT (OAuth, Cognito) ou identidade IAM.

3 Guarda as credenciais

O Identity injeta OAuth, chave de API ou IAM em cada ferramenta. Nada no código do agente.

4 Filtra ferramentas

Busca semântica: o agente recebe só as poucas relevantes para o pedido.

5 Intercepta

Uma Lambda antes e outra depois de cada chamada: validar, mascarar dados, controle fino.

6 Aplica regras

Policy confere cada chamada, e tudo aparece no trace do Observability.

O GANHO

Escreve uma vez, todo agente usa

Segurança num ponto só

Zero credencial no agente

Auditoria de cada chamada

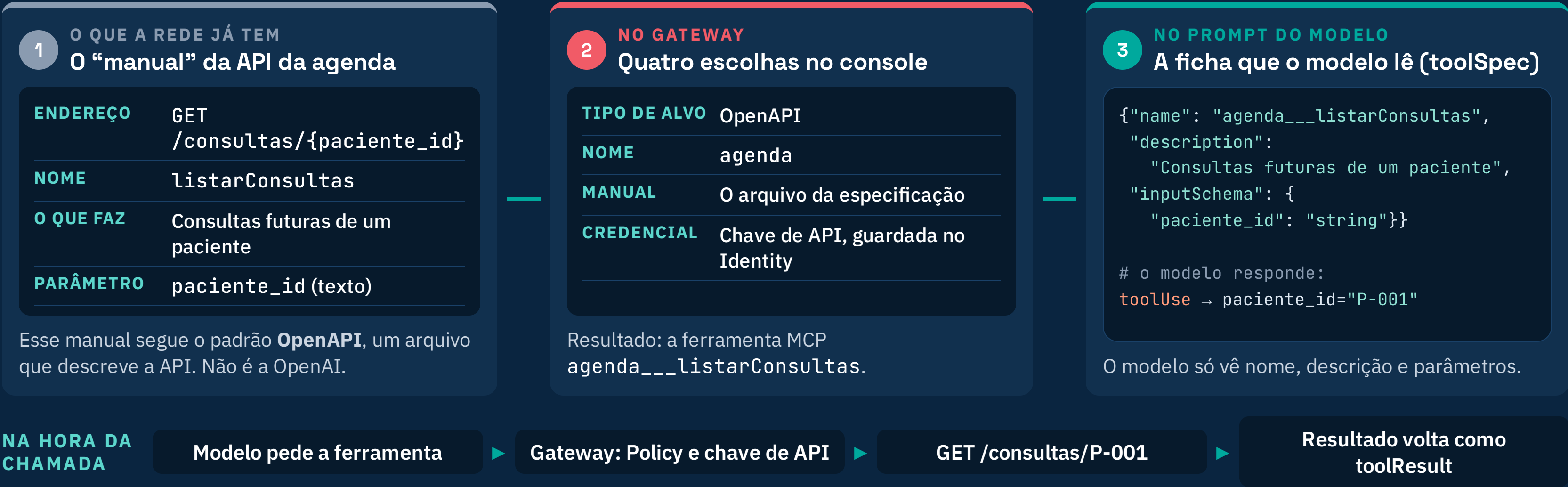
AGENTCORE GATEWAY NA PRÁTICA

Quatro casos na Rede

Agenda da Rede	Chamados de TI	Indicadores	Dado de paciente
HOJE Já existe uma API REST da agenda.	HOJE O service desk usa Jira.	HOJE Central, faturamento e diretoria querem números.	HOJE A resposta da ferramenta traz CPF e telefone.
COM O GATEWAY Sobe a especificação OpenAPI e ela vira ferramenta.	COM O GATEWAY Integração pronta do Gateway.	COM O GATEWAY Uma Lambda de SQL só leitura, num endereço só.	COM O GATEWAY Interceptor de resposta mascara antes do modelo.
GANHO Sem reescrever a API.	GANHO Sem código de integração.	GANHO Uma ferramenta, vários agentes.	GANHO LGPD fora do prompt.

AGENTCORE GATEWAY · EXEMPLO REAL

Da API da agenda ao prompt do modelo



Um lugar seguro para o agente rodar



Sessões isoladas

Cada conversa roda em sua própria microVM. O que acontece na sessão de um usuário não vaza para outra.



Qualquer framework

Strands, LangGraph, OpenAI Agents SDK, código próprio. Você empacota, o Runtime executa e escala.



Paga pelo uso ativo

Cobrança por consumo de CPU e memória. Enquanto o agente espera o modelo responder, a CPU ociosa não é cobrada.

É o “serverless” de setembro aplicado a agentes: sem servidor para gerenciar, com sessões que podem durar horas.

MEMORY

Lembrança de curto e de longo prazo

CURTO PRAZO

A conversa de agora

Guarda os eventos da sessão, para o agente não perder o fio.

"A Helena pediu para remarcar e ainda não escolheu o horário."

LONGO PRAZO

O que vale para a próxima

Extraí fatos, preferências e resumos, e os recupera em conversas futuras.

"Prefere consultas de manhã. Convênio alterado em setembro."



Memória de longo prazo é dado pessoal armazenado. O que guardar, por quanto tempo e como apagar são decisões de LGPD, não de engenharia.

POLICY

A regra sai do prompt e vira uma trava

UMA VEZ, QUANDO VOCÊ CRIA A REGRA

US\$ 0,13 por 1.000 tokens, só aqui

Você escreve em português

“Só remarque consultas do próprio paciente.”

A AWS traduz

Um modelo converte o texto numa política formal (Cedar) e confere se ela não ficou larga ou apertada demais.

Você revisa e publica

A política vira código, com versão.

A CADA USO DE FERRAMENTA, NO GATEWAY

US\$ 0,000025 por checagem

O agente pede

remarcar_consulta(C-1001)

O Policy confere

Checagem exata da regra: sem modelo, sem tokens, sempre com a mesma resposta.

Permitido ou negado

Executa, ou o agente recebe o motivo.

STRANDS AGENTS

0 loop inteiro em poucas linhas

```
from strands import Agent
from strands.models import BedrockModel

agente = Agent(
    model=BedrockModel(model_id="global.anthropic..."),
    system_prompt="Você é o agente da central...",
    tools=[buscar_consultas, buscar_horarios,
           consultar_preparo, remarcar_consulta],
)

agente("Preciso remarcar minha consulta...")
```

Aberto e da AWS

SDK de código aberto, em Python e TypeScript.
É o que roda na demonstração.

0 loop já vem pronto

Pensar, agir, observar, limites de voltas, MCP e múltiplos agentes.

Não é obrigatório

O AgentCore roda qualquer framework. Strands é só o caminho mais curto.

HARNESS

O loop continua existindo. Quem roda é a AWS.

COM CÓDIGO · STRANDS NO RUNTIME

- 1 Você escreve o agente**
O Strands já traz o loop; você controla cada detalhe.
- 2 Empacota e publica**
No Runtime do AgentCore, com a CLI.
- 3 Chama o endpoint**
Uma sessão isolada por conversa.

Para lógica própria, vários agentes, fluxos especiais.

COM O HARNESS · CONFIGURAÇÃO

- 1 Você declara**
Modelo, instruções, ferramentas, memória e skills.
- 2 A AWS roda o loop**
Pensar, agir e observar, numa sessão isolada, com cada passo no trace.
- 3 Você chama o harness**
Uma chamada à API, com a mensagem do usuário.

Precisa de mais controle? Um comando exporta o código Strands.

- !** Sem código de orquestração e sem taxa própria: você paga o que usar (Runtime, modelo, ferramentas). Bom para protótipos e times sem desenvolvedor dedicado.

DO CÓDIGO À NUVEM

Quatro comandos entre a sua máquina e o Runtime

agentcore create

Cria o projeto: framework, modelo, memória.

agentcore dev

Roda na sua máquina, com um inspetor de cada passo.

agentcore deploy

Empacota e publica no Runtime da sua conta.

agentcore invoke

Chama o agente já na nuvem.

O que o deploy faz

Empacota o código (com ou sem container) e guarda na sua conta.

O que você ganha

Um endpoint para a aplicação chamar, com uma sessão isolada por conversa.

Onde instalar

A CLI vem pelo npm: pacote @aws/agentcore, GA desde março de 2026.

INTERVALO • 20 MINUTOS

Na volta: vários agentes, e o humano no comando.

Orquestração, confiança e o Kahoot.

04

Orquestração

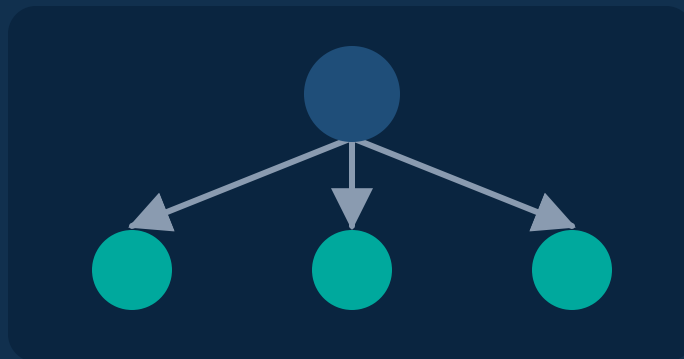
Quando um agente não basta, e onde o humano entra.

20 MINUTOS

PADRÕES

Quatro formas de organizar vários agentes

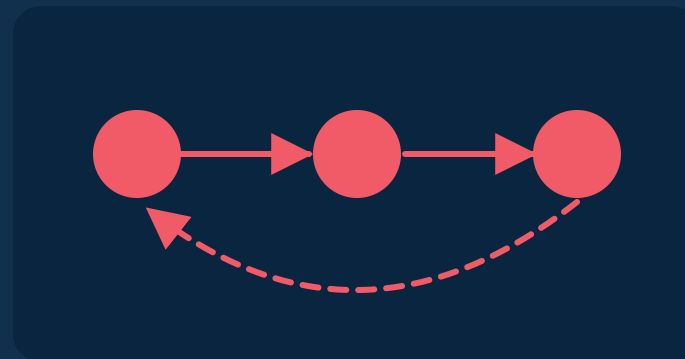
Supervisor



Um agente coordena e chama os especialistas como se fossem ferramentas.

O mais comum. Comece por ele.

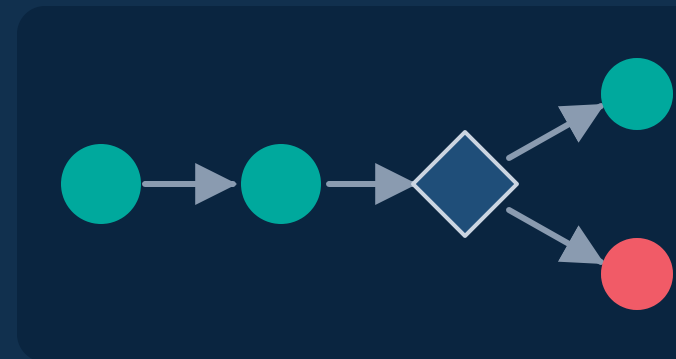
Revezamento



Os agentes se revezam, compartilhando o contexto. Também chamado de swarm.

Tarefas criativas e exploratórias.

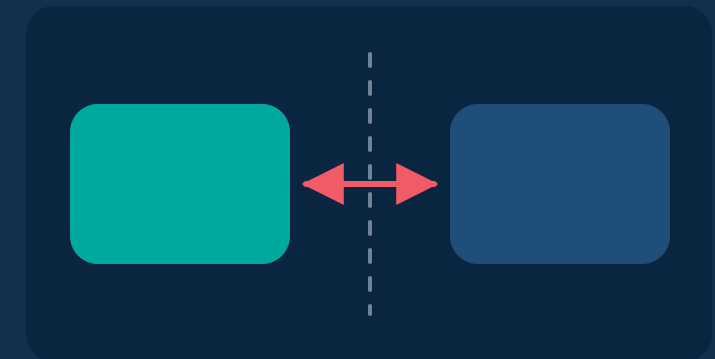
Grafo



O caminho entre os agentes é desenhado por você, com desvios condicionais.

Processos regulados e auditáveis.

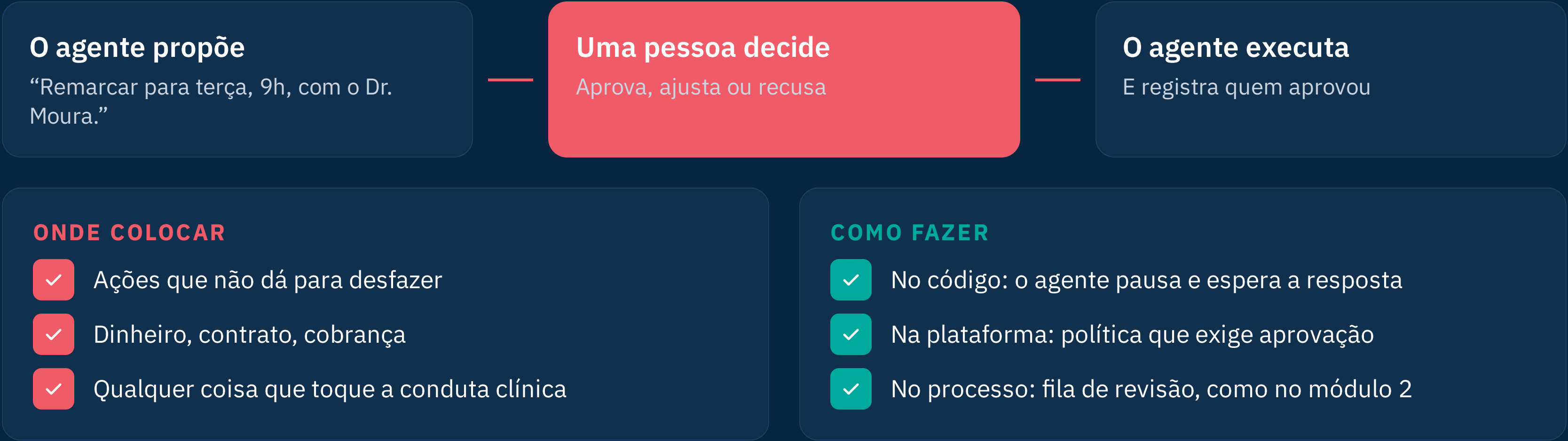
A2A



Agentes de times ou fornecedores diferentes conversam por um padrão comum.

Integração entre áreas e parceiros.

O freio de mão fica com uma pessoa



DESENHE COMIGO

E se a central virasse uma equipe?

O PEDIDO DA DONA HELENA

“Preciso remarcar a cardiologia de quinta. O exame de sangue precisa de jejum? Meu convênio cobre?”



Central • supervisor

Entende o pedido, chama os especialistas e junta as respostas.



Agenda

Consultas, horários e remarcação.

PACIENTE CONFIRMA

Orientações

Preparo de exames, com RAG.

SÓ LEITURA

Convênio

Cobertura e autorização, de outro time.

A2A

VALE DIVIDIR?

Hoje, não.

Cinco ferramentas cabem num agente só. É assim na demo.

DIVIDA QUANDO

- Entra uma área com dono próprio, como convênios.
- O agente erra a ferramenta entre dezenas delas.
- O prompt tem regras que brigam entre si.

QUANDO HÁ VÁRIOS AGENTES

Cinco jeitos de uma equipe de agentes falhar

FALHA	NA CENTRAL DA DONA HELENA	COMO EVITAR
1 Telefone sem fio	A Central resume o pedido e a Agenda recebe “remarcar cardiologia”, sem a quinta-feira.	Passe dados, não resumos: o id da consulta, a data, o paciente.
2 Pingue-pongue	A Agenda devolve para a Central, que devolve para a Agenda, sem fim.	Limite de passagens e de voltas por pedido.
3 Ação em dobro	Dois agentes remarcam a mesma consulta.	Só um agente tem a ferramenta que altera, e ela recusa repetição.
4 Aprovação perdida	A paciente confirmou para a Central, mas quem remarca não sabe disso.	A aprovação viaja junto com o pedido e fica registrada.
5 Conta multiplicada	Um pedido vira seis chamadas ao modelo, com o tempo de resposta somado.	Meça custo e tempo por pedido, não por agente.
! Todas respondem “200 OK”. Para enxergar, é preciso seguir o caminho inteiro do pedido: é o próximo bloco.		

05

Confiança

Observar, avaliar e limitar: o que separa demonstração de produção.

20 MINUTOS

RISCOS E CONTROLES

Cada risco tem um controle, e um endereço

Risco	Controle principal	Onde mora na AWS
1 · Chamado sem autorização	Aprovação humana + Policy	AgentCore Policy e a pausa do Strands (bloco 4)
2 · Ferramenta errada	Avaliação + descrição melhor	AgentCore Evaluations: acerto na escolha da ferramenta
3 · Ferramenta com erro	Fallback: tentar de novo, avisar, encaminhar	O código da ferramenta + o trace no Observability
4 · Pedido proibido	Guardrails + regras no prompt	Bedrock Guardrails, vistos ontem
5 · Preso em loop	Limites de voltas, tempo e custo	Limite no Strands ou no harness + alarme no CloudWatch
6 · Instrução escondida	Guardrails + menor privilégio	Filtro de ataque de prompt + IAM e AgentCore Identity

E todos eles só aparecem se alguém estiver olhando: o assunto dos próximos slides.

NOVOS TIPOS DE ERRO

Um agente erra diferente de um sistema comum

SOFTWARE TRADICIONAL

Erra com barulho: exceção, código 500, alerta no painel.

AGENTE

Erra em silêncio: responde com educação, em bom português, e está errado.

**Ferramenta errada**

Consultou exames quando era consulta.

**Parâmetro errado**

Buscou “cardiologista” em vez de “cardiologia”.

**Parou cedo**

Respondeu sem conferir o preparo.

**Inventou**

Citou um horário que a agenda não devolveu.

**Ficou em loop**

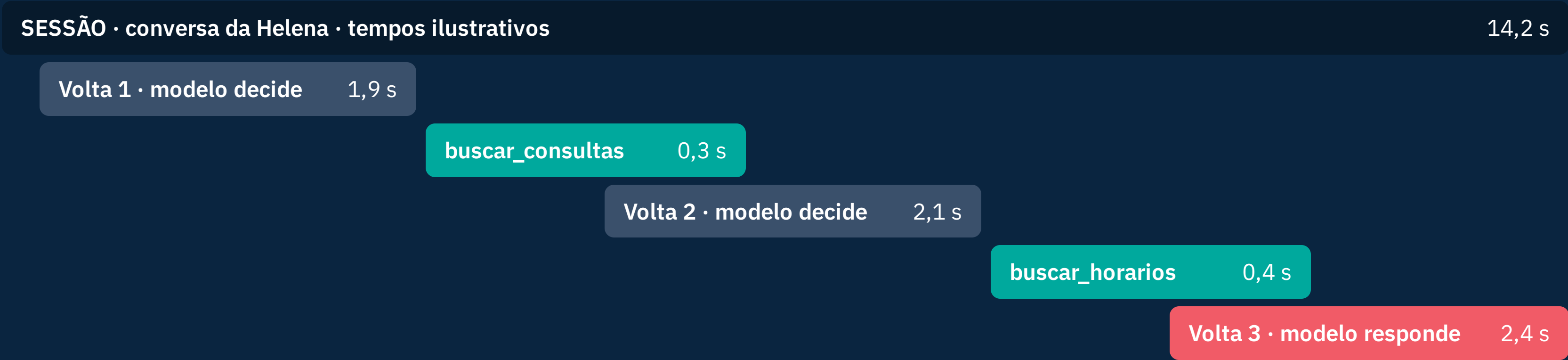
Repetiu a mesma busca dez vezes.

**Foi manipulado**

Seguiu uma instrução escondida em um texto.

Você já viu em setembro: CloudTrail e CloudWatch

0 raio-x de cada conversa



O que fica registrado
Cada decisão do modelo, cada ferramenta, entradas, saídas, tokens e tempo.

Onde aparece
No CloudWatch, em um painel próprio para IA generativa, no padrão aberto OpenTelemetry.

Para que serve
Achar o passo lento, o passo caro e o passo errado. E provar, depois, o que o agente fez.

EVALUATIONS

Uma nota para o agente, o tempo todo

Nível	A pergunta	Avaliadores prontos, por exemplo
Conversa inteira	O objetivo do usuário foi atingido?	Taxa de sucesso do objetivo
Cada resposta	Está correta, fiel às fontes, útil e segura?	Correção, fidelidade, utilidade, conteúdo nocivo
Cada ferramenta	Escolheu a ferramenta certa, com os parâmetros certos?	Acerto na escolha e nos parâmetros da ferramenta

Em produção

Avalia uma amostra das conversas reais, continuamente, e gera alerta quando a nota cai.

Sob demanda

Roda contra o seu gabarito a cada mudança de prompt, modelo ou ferramenta.

Sob medida

Avaliadores próprios: um modelo juiz com os seus critérios ou uma função com regra exata.

DEFESA EM CAMADAS

Três proteções em volta do agente

 Você viu ontem: Guardrails na entrada e na saída

3 · HUMANO NO LOOP · aprova as ações críticas

2 · POLICY · regra exata a cada uso de ferramenta, fora do prompt

1 · GUARDRAILS · filtra conteúdo, dados pessoais e tentativas de manipulação

AGENTE · prompt com regras claras e ferramentas de menor privilégio

Nenhuma camada basta sozinha

O prompt pode ser ignorado, o filtro pode deixar passar, a pessoa pode se distrair. Juntas, as chances caem muito.

Tudo isso já tem dono

Guardrails vimos ontem. Policy e aprovação humana, hoje. Falta só ligar.

O agente saiu do trilho. E agora?

COMO VAI SER

- 1 Abra kahoot.it no celular**
Ou use o app do Kahoot.
- 2 Digite o PIN da tela**
Pode jogar sozinho ou em dupla.
- 3 Dez perguntas rápidas**
De 20 a 30 segundos cada.
- 4 Depois de cada uma: por quê?**
Meio minuto de conversa sobre a resposta certa.

O QUE CAI

- Qual degrau?
- Onde entra o humano
- Agente preso em loop
- Instrução escondida
- Policy
- Observability
- harness

Casos da Rede: resumo de plantão, central de TI, reinício de serviço, dúvida de jejum e troca de convênio.

! Nenhum controle resolve sozinho. E todo risco só aparece se alguém estiver olhando.

DEMONSTRAÇÃO AO VIVO · 65 MINUTOS

Conteúdo na prática

Tudo o que vimos hoje, testado ao vivo na central.

Loop

Ferramentas

Humano no loop

Limites

Riscos

AgentCore

DEVER DE CASA · PARA O CHECKPOINT TÉCNICO

Leve o dia de hoje para o seu acelerador

Em grupo, evolua o desenho da semana 1 respondendo a quatro perguntas. O que vimos ontem (modelo, prompt, RAG e Guardrails) continua valendo.

BLOCO 1

1

Precisa de agente?

Qual degrau da escada? Por que não o de baixo?

BLOCO 2

2

Quais ferramentas?

O que só consulta e o que altera algo, com nome e descrição.

BLOCO 3

3

Onde roda?

Quais peças do AgentCore você usaria, ou o harness.

BLOCOS 4 E 5

4

Quem confere?

Onde entra o humano, que travas e dez casos de teste.

!

Concluiu que é degrau 1 ou 2? Resposta certa: o desenho vale do mesmo jeito, só sem agente.

PRÓXIMO ENCONTRO

Checkpoint Técnico • online

OBJETIVO

Acompanhar o desenvolvimento da solução e destravar dúvidas ou bloqueios para o avanço.

1

Avanços

Leve o que já desenvolveu na sua solução.

2

Evolução

Mostre o que mudou desde o módulo 3.

3

Dúvidas

Tire o que falta para concluir o desafio com sucesso.

4

Próximos passos

Mostre o caminho até a conclusão.

QUANDO

26/10 a 30/10

Última semana de outubro

03/11 a 06/11

Primeira semana de novembro

COMO AGENDAR

Escreva para gustavo.resende@a3data.com.br e escolha um horário.

PARA CONSULTAR DEPOIS

Material de apoio

Guia do Amazon Bedrock AgentCore

docs.aws.amazon.com/bedrock-agentcore

Preços do AgentCore

aws.amazon.com/bedrock/agentcore/pricing

Exemplos oficiais do AgentCore

github.com/aws-labs/agentcore-samples

Strands Agents: documentação e exemplos

strandsagents.com

Bedrock Guardrails

docs.aws.amazon.com/bedrock · [guardrails](#)

Building effective agents (Anthropic)

anthropic.com/engineering/building-effective-agents

Padrões de IA agêntica na AWS

docs.aws.amazon.com/prescriptive-guidance · [agentic-ai-patterns](#)

Caso Rede Mater Dei no blog da AWS Brasil

aws.amazon.com/pt/blogs/aws-brasil

Model Context Protocol

modelcontextprotocol.io

Projeto da demonstração

[agente-agendamento](#) · [README com o passo a passo](#)

Obrigado.

Conte o que achou do Módulo 3: conteúdo, ritmo e dinâmicas.

Próximo encontro: Checkpoint Técnico, online. Leve o desenho da solução.

